

Fondamenti di Informatica

A.A. 2016-2017

Prova intermedia del 7 giugno 2017 – Testo 1

NOME _____ COGNOME _____

Durante l'esame non si possono usare appunti né altro materiale.

Esercizio 1

Si dica se ciascuna delle seguenti affermazioni relative alle *classi* è corretta:

Sì No

- | | | |
|--------------------------|--------------------------|--|
| ☐ | ☐ | Una classe definisce un tipo di dato personalizzato, specificato dal programmatore. |
| ☐ | ☐ | L'insieme degli attributi e dei metodi di una classe è anche detto interfaccia della classe. |
| ☐ | ☐ | Per impostazione predefinita gli attributi e i metodi di una classe sono pubblici. |
| ☐ | ☐ | Gli operatori sono metodi che elaborano gli attributi di un oggetto. |
| ☐ | ☐ | Una classe può avere un solo costruttore con parametri. |
| ☐ | ☐ | I modificatori sono usualmente implementati come metodi costanti. |
| ☐ | ☐ | Una variabile del tipo definito dalla classe è detta oggetto. |
| ☐ | ☐ | In una classe, l'operatore di assegnamento deve essere sempre sovraccaricato |
| ☐ | ☐ | Le funzioni amiche non possono accedere ai membri privati di una classe. |
| ☐ | ☐ | Una classe può avere un solo costruttore di default. |

Esercizio 2

Scrivere le istruzioni C++ necessarie per allocare *dinamicamente* le seguenti variabili:

1. Un numero intero:

2. Un array di 150 numeri reali:

3. Una stringa di 64 caratteri alfanumerici:

4. Un array di 50 puntatori a numeri interi:

Esercizio 3

Completare le seguenti affermazioni:

1. L'insieme dei nomi e dei tipi delle operazioni disponibili in un tipo di dato astratto è detta _____.
2. La politica di gestione di una pila è detta _____.
3. Il _____ è invocato automaticamente per distruggere un oggetto di una classe.
4. Una lista concatenata semplice termina con un _____.
5. I _____ sono metodi che accedono agli attributi di un oggetto e ne restituiscono il valore.
6. Una _____ è una lista nella quale gli elementi si inseriscono solo in coda e si rimuovono solo in testa.

Esercizio 4

Identificare e correggere gli errori nella seguente serie di istruzioni:

```
1     template <type T>
2     T* test(T b, T s, int n) {
3         T* p = new T[n];
4         for (int k = 0; k < n; k++)
5             p[k] = b + k * s;
6         return *p; }
```

Soluzione:

**Il seguente esercizio dovrà essere svolto al calcolatore e consegnato su AulaWeb.
Accertarsi che la consegna sia confermata dal sistema!**

Esercizio 5

Allo scopo di sviluppare un software per la geometria analitica si realizzi in linguaggio C++ il template di classe *Retta*, parametrizzata sul tipo T ed avente per attributi il coefficiente angolare *_m* e l'intercetta *_q* (entrambi di tipo T) di una retta sul piano cartesiano, secondo l'equazione $y = mx + q$. Si implementino, inoltre, i seguenti metodi:

- Il costruttore di default che inizializzi il valore di *_m* e di *_q* a zero, utilizzando la lista di inizializzazione.
- Il costruttore con parametri che riceva come parametri il valore del coefficiente angolare *m* e dell'intercetta *q* (entrambi di tipo T) ed inizializzi i corrispondenti attributi della classe usando la lista di inizializzazione.
- Il costruttore di copia.
- Il distruttore.
- I selettori (un metodo per ciascun attributo).
- I modificatori (un metodo per ciascun attributo).
- Il metodo *inserisciDati* per leggere da tastiera i valori del coefficiente angolare *_m* e dell'intercetta *_q*. Il metodo non restituisce alcun valore di ritorno.
- Il metodo *stampaDati* che stampi a video l'equazione della retta secondo il seguente formato:
 $r: y = mx + q;$
dove a *m* viene sostituito il valore corrente di *_m* e a *q* il valore corrente di *_q*. Il metodo non restituisce alcun valore di ritorno.
- Il metodo *calcolaDistanza* che riceva come parametri le coordinate (x_P, y_P) di un punto P (due valori di tipo T), calcoli e restituisca come valore di ritorno la distanza *d* di P dalla retta (un numero reale), calcolata come:

$$d = \frac{|y_P - (mx_P + q)|}{\sqrt{1 + m^2}}$$

Si sviluppi, infine, la funzione *main* che operi nel modo seguente:

1. Dopo aver chiesto all'utente di inserire da tastiera un numero intero *n* e avere verificato che *n* sia positivo, allochi dinamicamente un array R di *n* oggetti di classe *Retta*, assegnando il tipo *double* al parametro T. Nel caso in cui *n* sia negativo, *main* terminerà stampando a video un messaggio di errore.
2. Scandisca l'array R e assegni i valori di *_m* e *_q* a ciascun elemento di R, chiamando il metodo *inserisciDati*.
3. Chieda all'utente di inserire le coordinate (x_Q, y_Q) di un punto Q (due numeri reali), scandisca l'array R, invochi su ciascun elemento dell'array R la funzione *calcolaDistanza* e ne stampi a video il valore di ritorno (ovvero la distanza del punto Q da ciascuna delle rette contenute nell'array R).
4. Deallochi l'array R e termini.

Soluzione

Esercizio 1 – Soluzione (1.5 punti, 0.15 per ogni risposta esatta)

Sì, Sì, No, Sì, No, No, Sì, No, No, Sì.

Esercizio 2 – Soluzione (1.0 punto, 0.25 per ogni risposta esatta)

1. `int* p = new int;`
2. `double* p = new double[150];`
3. `char* p = new char[65];`
4. `int** p = new int*[50];`

Esercizio 3 – Soluzione (1.5 punti, 0.25 per ogni risposta esatta)

Interfaccia, LIFO (Last In First Out), distruttore, puntatore nullo, selettori, coda.

Esercizio 4 – Soluzione (1.0 punto, 0.5 per ogni risposta esatta)

Riga 1 - sostituire type con typename: `template <typename T>`

Riga 6 - rimuovere *: `return p;`

Esercizio 5 – Soluzione (6.0 punti, 1.0 – 0.75 – 0.25 – 0.5 – 0.5 – 1.0 – 0.5 – 1.5)

```
#include <iostream>
#include <cmath>
using namespace std;

template <typename T>
class Retta {
public:
    Retta();
    Retta(T m, T q);
    Retta(const Retta<T>& r);
    ~Retta();
    T getM() const;
    T getQ() const;
    void setM(T m);
    void setQ(T q);
    void inserisciDati();
    void stampaDati() const;
    double calcolaDistanza(T xp, T yp) const;
private:
    T _m;
    T _q;
};

template <typename T>
Retta<T>::Retta() : _m(0), _q(0)
{
}

template <typename T>
Retta<T>::Retta(T m, T q) : _m(m), _q(q)
{
}

template <typename T>
Retta<T>::Retta(const Retta& r)
    : _m(r._m), _q(r._q)
{
}

template <typename T>
Retta<T>::~Retta() {}

template <typename T>
T Retta<T>::getM() const {
    return _m;
}

template <typename T>
T Retta<T>::getQ() const {
    return _q;
}

template <typename T>
void Retta<T>::setM(T m) {
    _m = m;
}

template <typename T>
void Retta<T>::setQ(T q) {
    _q = q;
}

template <typename T>
void Retta<T>::inserisciDati() {
    cout << "Inserire il coefficiente angolare: ";
    cin >> _m;
    cout << "Inserire l'intercetta: ";
    cin >> _q;
}

template <typename T>
void Retta<T>::stampaDati() const {
    cout << "r: y = " << _m << "x + " << _q << ";
    << endl;
}

template <typename T>
double Retta<T>::calcolaDistanza(T xp, T yp) const
{
    return abs(yp - (_m * xp + _q)) /
        sqrt(1 + _m * _m);
}

int main()
{
    int n = 0;
    cout << "Inserire dimensione: ";
    cin >> n;
    if (n <= 0) {
        cout << "Errore" << endl;
        return -1;
    }

    Retta<double>* p = new Retta<double>[n];
    if (p == NULL) return -2;
    for (int i = 0; i < n; i++) {
        p[i].inserisciDati();
    }

    double xq = 0.0, yq = 0.0;
    cout << "Inserire coordinate: ";
    cin >> xq >> yq;
    for (int k = 0; k < n; k++)
        cout << p[k].calcolaDistanza(xq, yq)
            << endl;

    delete[] p;

    return 0;
}
```